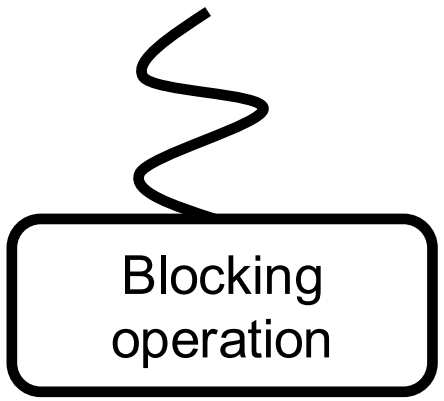
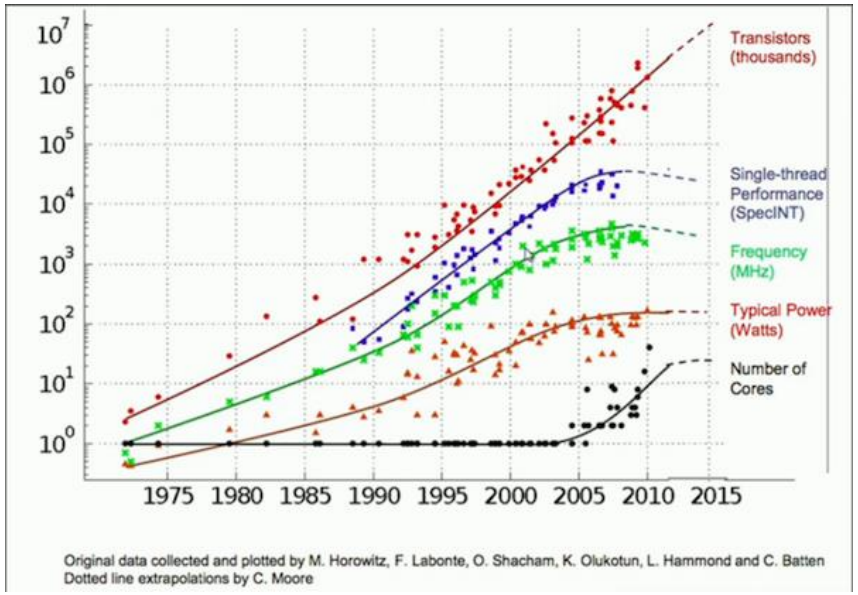
Concurrency







Do threads share the same Code segment?

PTBR?	Yes
Registers?	No
Inst Pointer?	No
Stack?	No
Heap?	Yes

Thread

User-level versus kernel-level threads

Context switching can happen at any time

Q: Possible increments to shared var?

Thread 1

```
mov 0x123, %eax
add $0x1, %eax
mov %eax, 0x123
```

Thread 2

```
mov 0x123, %eax
add $0x2, %eax
mov %eax, 0x123
```

Timeline View

Thread 1

mov 0x123, %eax

add $0x1, %eax

mov %eax, 0x123

Thread 2

mov 0x123, %eax

add $0x2, %eax

mov %eax, 0x123

How much is added to shared variable?

3: correct!

Timeline View

Thread 1

mov 0x123, %eax

add $0x1, %eax

mov %eax, 0x123

Thread 2

mov 0x123, %eax

add $0x2, %eax

mov %eax, 0x123

How much is added?

2: incorrect!

Timeline View

Thread 1

mov 0x123, %eax

add $0x1, %eax

mov %eax, 0x123

Thread 2

mov 0x123, %eax

add $0x2, %eax

mov %eax, 0x123

How much is added?

1: incorrect!

Timeline View

Thread 1

mov 0x123, %eax

add %0x1, %eax

mov %eax, 0x123

Thread 2

mov 0x123, %eax

add %0x2, %eax

mov %eax, 0x123

How much is added?

3: correct!

Timeline View

Thread 1

mov 0x123, %eax

add %0x1, %eax

mov %eax, 0x123

Thread 2

mov 0x123, %eax

add %0x2, %eax

mov %eax, 0x123

How much is added? 2: incorrect!

Non-Determinism

Concurrency leads to non-deterministic results

- Not deterministic result: different results even with same inputs
- race conditions: “races” between instructions executing on shared data can result in non-deterministic results, depending on order

Whether bug manifests depends on CPU schedule!

Passing tests means little

How to program: imagine scheduler is malicious

Assume scheduler will pick bad ordering at some point…

What do we want?

Want instructions to execute as an uninterruptable group

That is, we want them to be atomic

```
mov 0x123, %eax
add %0x1, %eax
mov %eax, 0x123
```

critical section:
Instructions accessing shared data

Need mutual exclusion for critical sections

- if process A is in the critical section, process B can't also be
- Okay and desirable if other processes do unrelated work

- A **critical section** is a piece of code that accesses a *shared* resource, usually a variable or data structure.
- A **race condition** (or **data race** [NM92]) arises if multiple threads of execution enter the critical section at roughly the same time; both attempt to update the shared data structure, leading to a surprising (and perhaps undesirable) outcome.
- An **indeterminate** program consists of one or more race conditions; the output of the program varies from run to run, depending on which threads ran when. The outcome is thus not **deterministic**, something we usually expect from computer systems.
- To avoid these problems, threads should use some kind of **mutual exclusion** primitives; doing so guarantees that only a single thread ever enters a critical section, thus avoiding races, and resulting in deterministic program outputs.

Synchronization

Build higher-level synchronization primitives within the OS

- Operations that ensure correct ordering of instructions across threads

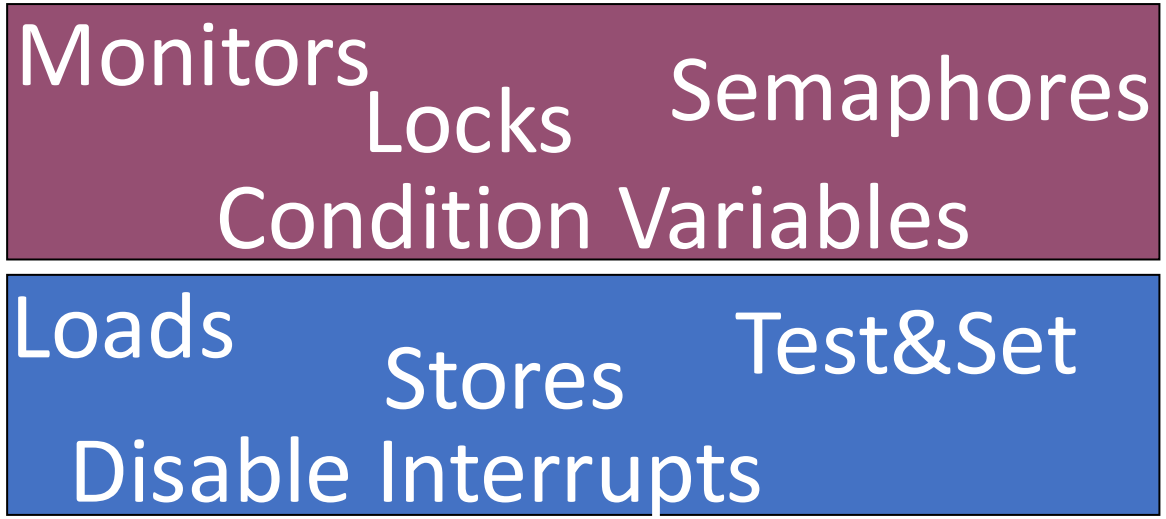
Why is this an OS (rather than app) concern?

Historically, major first user of concurrency

Often need hardware and OS support

Some efficiency concerns addressed through OS support

Build it once and get it right



Locks

Goal: Provide mutual exclusion (mutex)

Three common operations:

- Allocate and Initialize
  - `Pthread_mutex_t mylock = PTHREAD_MUTEX_INITIALIZER;`
- Acquire
  - Acquire exclusion access to lock;
  - Wait if lock is not available (some other process in critical section)
  - Spin or block (relinquish CPU) while waiting
  - `Pthread_mutex_lock(&mylock);`
- Release
  - Release exclusive access to lock; let another process enter critical section
  - `Pthread_mutex_unlock(&mylock);`

Correct mutual exclusion for `balance`

```
1  pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
2
3  Pthread_mutex_lock(&lock); // wrapper; exits on failure
4  balance = balance + 1;
5  Pthread_mutex_unlock(&lock);
```

```
lock(&mutex);
balance = balance + 1;
unlock(&mutex);
```

Other Examples

Consider multi-threaded applications that do more than increment shared balance

Multi-threaded application with shared linked-list

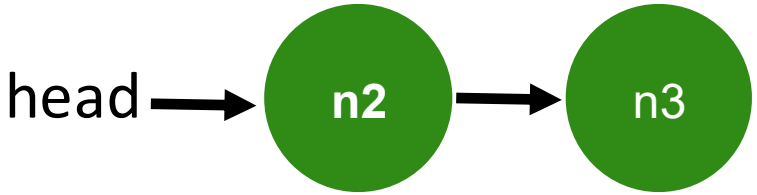
- All concurrent:
  - Thread A inserting element a
  - Thread B inserting element b
  - Thread C looking up element c
  - Assume no delete’s for now

Shared Linked List

```
typedef struct __node_t {
        int key;
        struct __node_t *next;
} node_t;

Typedef struct __list_t {
        node_t *head;
} list_t;

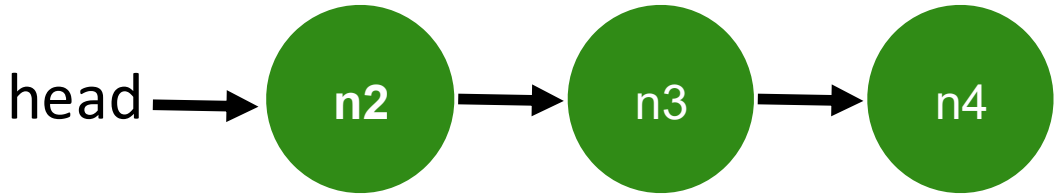
Void List_Init(list_t *L) {
        L->head = NULL;
}
```



Shared Linked List

```
Void List_Insert(list_t *L, int key) {
        node_t *new = malloc(sizeof(node_t));
        assert(new);
        new->key = key;
        new->next = L->head;
        L->head = new;
}

int List_Lookup(list_t *L,  int key) {
        node_t *tmp = L->head;
        while (tmp) {
                if (tmp->key == key)
                        return  1;
                tmp = tmp->next;
        }
return 0;
}
```



What can go wrong?
Find a schedule that leads to problem?